

PDFa11yMut: Measuring Mutation-Specific Detection by PDF Accessibility Validators

Author information withheld in this working draft

Abstract—Automated PDF accessibility checkers provide useful conformance evidence, but a clean report is not a complete accessibility oracle. PDFa11yMut measures mutation-specific checker behavior with paired transformations: a reference-suite baseline is mutated at the structure level, the primary verifier checks the intended delta, MuPDF independently checks visual invariants and operator-aware normalized structural deltas, and hash-linked validator evidence is recorded. Formal mutations provide positive cases; 18 no-op and benign controls support descriptive, baseline-relative sanity checks, not general specificity estimates. Formal rows retain paired baseline/mutant classifications and adjudication records; no independent-human coding claim is made. Results remain mutation-specific detection/kill behavior rather than general accessibility accuracy. The artifact separates conformance-oriented mutations from semantic or assistive-representation mutations. The current scope contains ten specified operators, nine reference PDFs, 70 valid verified mutants, 69 active mutants, 207 active formal validator rows, 18 completed controls, and nine illustrative exploratory NVDA observations.

Index Terms—PDF accessibility, PDF/UA, mutation testing, validator evaluation, assistive technology, reproducibility

I. Introduction

PDF accessibility evaluation combines syntactic constraints with document-dependent judgments. PDF/UA and the Matterhorn Protocol provide an important conformance model, while accessible-PDF techniques provide atomic examples and procedures. These resources do not make every question about meaning, logical order, or useful alternative text mechanically decidable. A validator result must therefore be interpreted relative to its rules, version, profile, and configuration.

Mutation testing makes this boundary observable. A controlled mutant starts from a known baseline, changes one specified property, and is retained only when the intended transformation and invariants are verified. The unit of evidence is the exact source hash, mutant hash, structural delta, rendering result, validator configuration, and optional assistive-technology observation. A mutant that survives a formal check is described as surviving that check; it is not automatically a validator false negative or evidence that the document is accessible.

PDFa11yMut operationalizes recognized PDF accessibility failure classes as reusable transformations. Its contribution is the evidence chain and operator-level analysis, not invention of heading, list, table, language, role-map, or alternate-description requirements. The framework makes the boundary between formal validation

and semantic/assistive representation observable through paired, hash-linked mutations.

II. Research Questions

- 1) How do fixed PDF accessibility checking configurations respond to verified formal accessibility mutations?
- 2) How do detection results vary by operator, document structure, baseline status, and validator capability?
- 3) Which semantics- or representation-dependent mutations survive automated checks, and what assistive-representation differences are observed in the selected exploratory AT sample?
- 4) When validators disagree on the same hash-linked mutant, which rule, configuration, baseline condition, or structural factor explains the disagreement?

III. Related Work and Scope

Mutation testing studies the behavior of test suites against controlled mutants, including mutation operators, killed/survived mutants, equivalent or ineffective mutants, and mutant validity [4]. Ma11y is the closest accessibility mutation-testing predecessor in the web domain, with WCAG-derived operators and an automated oracle [2]; Android accessibility mutation testing supplies a mobile precedent [3]. PDFa11yMut transfers that experimental idea to tagged-PDF structure while retaining the need for format-aware oracles.

Kumar, Padath, and Wang provide a PDF accessibility benchmark with expert-validated criteria and systematically manipulated variants; PDFa11yMut is complementary because its ground truth is an explicit low-level transformation with a structural oracle rather than an expert label alone [1]. The PDF/UA-1 Reference Suite provides reference-quality tagged documents across document types; nine of its files are used here as baselines, while the missing 2-07 item is documented rather than reconstructed [5]. The PDF Association’s Matterhorn Protocol and Techniques for Accessible PDF provide normative failure conditions, atomic PASS/FAIL examples, and test procedures; PDFa11yMut does not claim those defect concepts as novel [6], [7]. The veraPDF corpus and Isartor suite are complementary sources for validator testing; Isartor targets PDF/A-1 archival conformance and deliberately violates archival requirements [9], [10]. A recent comparison of PDF/UA-1 validators reports matching, partial, and divergent outcomes; our paired

mutants add a hash-linked transformation unit but do not rank tools [8].

The gap addressed here is narrower: reusable mutation of existing reference structures with explicit preconditions, allowed deltas, invariants, and paired baseline/mutant validator evidence. Atomic bad PDFs and validator comparisons remain important complementary resources, not substitutes for this mutation-testing unit.

IV. Experimental Model

Class A contains properties with a documented machine-checkable or profile-specific conformance condition in this study: M03 incorrect heading level, M07 missing figure alternate description, M08 missing document language, M09 invalid RoleMap target, and M10 illegal table-child role. Class B contains properties for which a structurally legal file can still expose an unintended semantic or assistive representation: M01 structural reading-order swap, M02 omission from assistive structure, M04 marked-content association swap, M05 internal marked-content sequence reversal, and M06 duplicate list-item reference. M06 is structurally verified but has no dedicated duplicate-reference failure condition in the reviewed normative materials. Class B survival is therefore reported as no automated finding, not as a validator failure.

The repository contains ten operator specifications with preconditions, transformations, invariants, expected structural deltas, expected assistive effects, and evidence fields. M04 mutants in the current corpus pass the association-only purity gate. The historical label G02-M04 is permanently excluded: its exact golden/mutant pair and complete COS-level audit trail are unavailable. Current hash-linked M04 mutants are separate artifacts and are interpreted only under the Class B semantic protocol.

V. Reproducible Pipeline

The input PDFs in corpus/golden/ are immutable working inputs. The command `python -m pdfallymut generate-all` applies all applicable operators, records source and mutant SHA-256 values, performs parseability/page/content/structure checks, and rasterizes both PDFs with Poppler `pdftoppm` at 150 DPI. The current visual rule is exact RGB pixel equality with tolerance zero. A failed invariant removes the output and records an exclusion.

The primary verifier uses `pypdf` and is separate from the validators under evaluation. A second `PyMuPDF`/`MuPDF` audit checks page/render invariants for every active pair and confirms operator-aware raw-COS deltas and whole-tree expected-result signatures for all 69 active pairs. The independent checks compare structure-tree order/reachability for M01/M02, marked-content association for M04, content-item order for M05, repeated direct /LI references for M06, and scoped structural deltas for M03 and M07–M10. Generator-side verification is not relabeled as independent evidence. `MuPDF`

rendering equality is checked at 72 DPI; the primary generation invariant remains the Poppler 150-DPI render. Per-mutant delta manifests record allowed changes, invariants, observed changes, purity status, and verification methods. Active result tables are rebuilt from canonical data by `scripts/rebuild_analysis.py`, while `scripts/audit_submission.py` checks evidence, hashes, mappings, denominators, and manifest consistency. The PDF and release are regenerated only after the final artifact state is fixed.

VI. Validator and AT Evidence

PAC Formal, Acrobat Full Check, and veraPDF evidence are stored in hash-addressed raw-report paths and compared with matching reference baselines. The active formal scope is 69 verified mutants and 207 validator-mutant rows, 69 per validator. Each row retains a baseline-relative classification, rationale, and adjudication record; coder identity and independent-human provenance are not part of the reported study claim. Three M10 disagreements were resolved under a decision rule fixed before final adjudication: the Acrobat Headers proxy applies only when fresh hash-linked verification proves an exact direct-child /TH to /P change under the same /TR, unchanged target content and non-target structure, equal page-content hashes, and pixel-identical rendering; the matched baseline must pass Headers and the mutant must newly fail it. G09-M08 is excluded because the PAC baseline rejects the golden’s language value while veraPDF treats the non-empty value as present. Four historical M09 artifacts remain excluded as `EXCLUDED_UNVERIFIABLE_TARGET`: their original mutant bytes are unavailable, and the reviewed Matterhorn rule applies to unused as well as used RoleMap entries, so the old unused-target rationale cannot establish an invalid mutation. Acrobat manual-check prompts are retained separately and are not automated detections. The canonical Acrobat configuration is the native-session setting of 31 of 32 checks; an older metadata field saying 31 of 31 is preserved as historical provenance.

The negative-control ledger contains 18 paired no-op and benign controls. Every control has an independently verified artifact plus native PAC PDF, Acrobat HTML, and veraPDF reports with hash-linked paths in `data/controls.csv`. These controls support baseline and false-positive analysis, but their native findings are interpreted only relative to the matched golden and benign pair; they are not positive-study results. In particular, both G05 Acrobat controls carry the same pre-existing “Lbl and LBody” failure and both G09 PAC controls carry the same Natural language failure.

PAC AI was executed as a separate configuration on a selected cohort of 34 mutants and matching golden baselines. The native PAC UI exposed aggregate counts for semantic panels but no finding identity, finding text, element-level attribution, or repro-

TABLE I
Formal validator outcomes in the active scope.

Validator	Detected	No target	Manual	Total	Rate
PAC	30	39	0	69	43.5%
Acrobat	26	28	15	69	48.1%
veraPDF	30	39	0	69	43.5%

ducible semantic export/API. All 34 rows therefore remain AI_AMBIGUOUS/REVIEW_REQUIRED; no PAC-AI detection or non-detection is reported. PAC AI is optional future work and is not required for completion of the formal study.

Nine selected cases were observed with NVDA 2026.2.0.57664 and Adobe Acrobat 26.2.21931.0 on Windows 10 Home 25H2. This is a qualitative, single-observer illustrative exploratory sample, not a representative pilot or user study. Case selection and convenience/coverage rationale are recorded in the artifact. Transcripts are linked to a preserved timestamped NVDA log; fresh paired Speech Viewer captures are preserved for M01 and a second case. The clean M01 rerun confirmed that the golden announced the logo first while the mutant announced the address block first; the historical instability is retained as context. M08 preserves the observed no-difference result. These observations are reported separately from validator detections.

VII. Results

A. Formal validator results

The active formal scope contains 69 verified mutants after the documented exclusions. The canonical data contain 207 classified validator-mutant rows (69 mutants x three validators); PAC AI and assistive-technology observations are not included in these formal outcomes.

Every displayed rate is Detected/(Detected + No target); the exact numerator and denominator are retained in the generated CSV. Acrobat manual checkpoints are retained separately and are not detections.

The active formal results show that all Class B mutations survived automated detection in this scope. This is a scoped observation about the tested configurations; semantic validity and assistive-technology impact require separate evidence.

The class-specific table separates formal conformance-oriented outcomes from semantic survival. In the active scope, Class A contains 30 mutants: PAC detected 30, Acrobat detected 26, and veraPDF detected 30. Class B contains 39 mutants and received no automated detection in the tested configurations; Acrobat produced 15 Class B manual checkpoints, which are not automated detections. The overall Acrobat count is therefore 26 automated detections, 28 automated no-target findings, and 15 manual-review rows; its displayed automated rate is 26/54, while PAC and veraPDF rates are 30/69. The Class B result

is a statement about the tested automated checks, not a claim that the semantic mutations are universally harmful or that a validator was obligated to identify them.

Pairwise agreement is also scope-dependent. PAC and veraPDF agreed on 69 of 69 active mutants (100%), while PAC and Acrobat agreed on 50 of 69 (72.5%) and Acrobat and veraPDF agreed on 50 of 69 (72.5%). These results are descriptive of the recorded builds and baselines; they do not rank validators globally. The machine-generated disagreement audit contains 38 pairwise disagreement rows. Thirty are Acrobat manual-review versus an automated decision, six are M09 RoleMap rule/profile differences, and two are other rule/profile differences. This decomposition keeps manual prompts, proxy/direct classification, baseline-carried findings, and target-specific automated findings distinct; it explains disagreement structure rather than presenting a global validator ranking.

VIII. Threats to Validity

The principal threats are corpus size and source clustering, operator selection and representativeness, multiple legal PDF encodings, parser-specific interpretation, validator version/profile differences, the study-specific direct-consequence proxy, PAC-AI instability and limited exportability, assistive-technology and viewer variability, the limited PDF/UA version scope, the unretained historical download archive, and visual-renderer limitations. One source document cannot establish an operator-wide effect. Visual equality cannot establish equivalent speech output. A screen-reader observation from one AT/viewer pair cannot establish universal behavior. The nine local PDFs are mapped to the PDF/UA Reference Suite under the collection-level CC BY 4.0 terms; the omitted 2-07 suite item and the absence of the original acquisition archive are documented in the artifact. The second-parser audit provides a common artifact-verification route, but it does not make the Class B properties machine-checkable or require validators to report them. The dedicated M06 audit confirms its structural delta but classifies it as semantic/Class B. The four historical M09 pairs cannot be re-audited because original mutant bytes are unavailable. Coding/adjudication records are retained as internal audit provenance but are not used to support an independent-coder claim. Negative controls remain descriptive paired evidence rather than a basis for general accuracy or specificity claims.

IX. Conclusion

PDFa11yMut’s defensible contribution is a paired, hash-linked methodology for measuring mutation-specific PDF accessibility checker behavior, with generator-side delta checks, independent rendering invariants, a scoped raw-COS audit, matched baseline evidence, capability-aware classification, descriptive controls, and separate assistive-representation observations. The formal validator phase, 18-control evidence set, and nine-case exploratory AT

TABLE II

Class-specific formal outcomes. Class B rows are reported as automated-check outcomes, not as claims that the validators should have detected semantic defects.

Class	Validator	Detected	No target	Manual	Total	Rate
Class A	PAC	30	0	0	30	100.0%
Class A	Acrobat	26	4	0	30	86.7%
Class A	veraPDF	30	0	0	30	100.0%
Class B	PAC	0	39	0	39	0.0%
Class B	Acrobat	0	24	15	39	0.0%
Class B	veraPDF	0	39	0	39	0.0%

TABLE III

Detection by operator and validator. A dash indicates that the rate denominator is zero.

Operator	PAC D/N	PAC rate	Acrobat D/N	Acrobat rate	veraPDF D/N	veraPDF rate	Class
M01	0/8	0%	0/0	–	0/8	0%	B
M02	0/9	0%	0/9	0%	0/9	0%	B
M03	8/0	100%	8/0	100%	8/0	100%	A
M04	0/8	0%	0/8	0%	0/8	0%	B
M05	0/7	0%	0/0	–	0/7	0%	B
M06	0/7	0%	0/7	0%	0/7	0%	B
M07	7/0	100%	7/0	100%	7/0	100%	A
M08	8/0	100%	8/0	100%	8/0	100%	A
M09	3/0	100%	0/3	0%	3/0	100%	A
M10	4/0	100%	3/1	75%	4/0	100%	A

sample are complete for the current 69-mutant scope, with AT evidence kept separate from detection rates. The M09 target-selection rule now requires a reachable structure element to use the mutated RoleMap key; four historical M09 artifacts remain excluded because their original mutant bytes are unavailable for re-audit. M06 is retained as a structurally verified Class B case rather than presented as a machine-checkable validator miss. Corpus filename mapping and collection-level licensing are documented in the artifact; the original acquisition archive/timestamp was not retained, and PAC AI is optional future work rather than a prerequisite for the formal study.

- [8] D. Kosovac, “Understanding PDF/UA Validation Results: A Comparative Study Across Tools,” PDF Association, Dec. 26, 2025, PDF Association validator comparison.
- [9] veraPDF, “veraPDF-corpus,” CC BY 4.0, <https://github.com/veraPDF/veraPDF-corpus>.
- [10] PDF Association, “Isartor Test Suite: Terms of Use,” <https://pdfa.org/wp-content/uploads/2011/08/Isartor-Test-Suite4.pdf>.

References

- [1] A. Kumar, T. Padath, and L. L. Wang, “Benchmarking PDF Accessibility Evaluation: A Dataset and Framework for Assessing Automated and LLM-Based Approaches for Accessibility Testing,” ASSETS 2025, doi: 10.1145/3663547.3746380.
- [2] M. Tafreshipour, A. V. Deshpande, F. Mehralian, I. Ahmed, and S. Malek, “Ma1ly: A Mutation Framework for Web Accessibility Testing,” ISSTA 2024, pp. 100–111, doi: 10.1145/3650212.3652113.
- [3] H. N. da Silva, S. R. Vergilio, and A. T. Endo, “Accessibility Mutation Testing of Android Applications,” Journal of Software Engineering Research and Development, vol. 10, 2022, doi: 10.5753/jserd.2022.2133.
- [4] Y. Jia and M. Harman, “An Analysis and Survey of the Development of Mutation Testing,” IEEE Transactions on Software Engineering, vol. 37, no. 5, pp. 649–678, 2011, doi: 10.1109/TSE.2010.62.
- [5] PDF Association, “PDF/UA-1 Reference Suite 1.1,” CC BY 4.0, <https://pdfa.org/resource/pdfua-reference-suite/>.
- [6] PDF Association, “Matterhorn Protocol 1.1,” 2021, <https://pdfa.org/download-area/publications/Matterhorn-Protocol-1-1.pdf>.
- [7] PDF Association, “Techniques for Accessible PDF,” 2025, <https://pdfa.org/techniques-for-accessible-pdf/>.